Análisis de algoritmos
Escuela Politécnica Superior, UAM, 2026–2027

Práctica n.º 2

Fecha de entrega de la práctica

- Grupos del Miércoles: 17 de noviembre de 2026.
- Grupos del Jueves: 18 de noviembre de 2026.

La práctica estudia experimentalmente el tiempo de ejecución y el número de comparaciones de dos algoritmos de ordenación basados en la estrategia de divide y vencerás: *MergeSort* y *QuickSort*. Se medirán con permutaciones de distintos tamaños y se compararán los resultados empíricos con las predicciones teóricas.
Si utilizáis una herramienta de IA como apoyo en algún apartado o cuestión, indicad en la memoria para qué la habéis empleado.

Algoritmo MergeSort

1. Implementad en `sorting.c` el algoritmo de ordenación *MergeSort* con los prototipos:

```
int MergeSort(int* array, int ip, int iu)
int merge(int* array, int ip, int iu, int im)
```

   Los índices `ip`, `iu` e `im` delimitan segmentos inclusivos: `ip` y `iu` son, respectivamente, el primer y el último índice del segmento, e `im` es el último índice de la mitad izquierda. Ordenad de menor a mayor el segmento indicado, sin modificar los elementos exteriores. Definid como operación básica (OB) la comparación entre dos claves. Ambas funciones devolverán `ERR` si se produce un error o el número de evaluaciones de la OB si terminan correctamente. El recuento de `MergeSort` debe incluir las operaciones de las llamadas recursivas y de las combinaciones. Notificad los errores de reserva de memoria.

   Adaptad `exercise4.c` de la práctica anterior para probar `MergeSort`. Comprobad que ordena correctamente.

2. Adaptad `exercise5.c` para medir `MergeSort` con permutaciones de distintos tamaños. Guardad para cada tamaño el número de permutaciones, el tiempo medio y los valores medio, mínimo y máximo de ejecuciones de la OB. Comparad los resultados con el análisis teórico y representadlos en gráficas con ejes, unidades y leyendas claramente identificados.

   Podéis consultar una IA para preparar un ajuste de curvas con el comando `fit` de GNUPlot. Elegid y justificad modelos acordes con el análisis teórico; por ejemplo, una función proporcional a $n \log n$ para el número de comparaciones de *MergeSort*. Ajustad por separado las operaciones básicas y el tiempo de CPU, y comprobad que el ajuste usa los datos medidos y no confunde el tiempo de pared con el tiempo de CPU. Incluid en la memoria el modelo, los parámetros y el intervalo de ajuste.

Algoritmo QuickSort

Las funciones de ordenación trabajarán sobre segmentos inclusivos de `array`, desde `ip` hasta `iu`, y ordenarán de menor a mayor. En todos los casos se contará como OB una comparación entre dos claves. Las funciones devolverán `ERR` si se produce un error o el número de ejecuciones de la OB si terminan correctamente; el índice final del pivote se comunicará mediante `pos`.
Para pasar la función de selección del pivote a `partition`, declarad en `sorting.h` el tipo:

```
typedef int (*pfunc_pivot)(int*, int, int, int*);
```

3. Implementad `QuickSort_min` con el prototipo `int QuickSort_min(int* array, int ip, int iu)`. Implementad también las funciones auxiliares:

```
int partition(int* array, int ip, int iu, int* pos, pfunc_pivot pivot)
int pivot_min(int* array, int ip, int iu, int* pos)
```

En esta primera variante, el pivote es el primer elemento del segmento: `pivot_min` escribirá `ip` en `*pos` y devolverá cero, ya que no compara claves. `partition` devolverá el número de comparaciones que realiza y `QuickSort_min` sumará los recuentos de la partición y de las llamadas recursivas. Adaptad `exercise4.c` para probar la función y comprobar que ordena correctamente, incluidos segmentos de uno o ningún elemento.

4. Implementad `QuickSort_mean` y su selector de pivote `pivot_mean` con los prototipos:

```
int QuickSort_mean(int* array, int ip, int iu)
int pivot_mean(int* array, int ip, int iu, int* pos)
```

`pivot_mean` elegirá el índice central del segmento, calculado de forma que se evite el desbordamiento de la suma de índices, lo escribirá en `*pos` y devolverá cero, pues no compara claves.

5. Implementad `pivot_median` y `QuickSort_median` con los prototipos:

```
int pivot_median(int* array, int ip, int iu, int* pos)
int QuickSort_median(int* array, int ip, int iu)
```

`pivot_median` elegirá como pivote la mediana de los valores situados en los índices `ip`, `ip + (iu-ip)/2` e `iu`. Contad las comparaciones de claves que realiza el selector además de las efectuadas durante la partición; el número de comparaciones del selector puede variar según los valores comparados. Modificad `partition` para sumar ambos recuentos sin contar dos veces ninguna comparación.

En un array de cinco elementos con índices de 0 a 4 y valores $(1, 3, 5, 4, 2)$, el selector debe devolver el índice 4, que contiene la mediana de los valores de los extremos y del centro: 1, 5 y 2.

6. Adaptad `exercise5.c` para medir `QuickSort_min`, `QuickSort_mean` y `QuickSort_median` sobre permutaciones de distintos tamaños. Guardad los resultados de cada variante por separado y generar las gráficas de los tiempos medios de CPU y los números medio, mínimo y máximo de ejecuciones de la OB. Ajustad el tiempo medio de CPU y el número medio de las OB. Para los ajustes de curvas, consultad las indicaciones del apartado de *MergeSort* y justificad las curvas utilizadas.

Cuestiones sobre la práctica

1. Comparad las tres estrategias de selección del pivote de *QuickSort* mediante gráficas de tiempo medio de CPU y de número medio de comparaciones. Explicad las diferencias observadas y relacionadlas con el análisis teórico.
2. Comparad el rendimiento empírico de *MergeSort* con el de la variante más rápida de *QuickSort* en vuestras mediciones. Tened en cuenta que el resultado puede depender de la implementación, del equipo y de las entradas utilizadas.
3. ¿Habéis observado picos en las gráficas? Proponed causas posibles y describid cómo comprobaríais si se deben al algoritmo, a la variabilidad de la medición o al entorno de ejecución.
4. Suponiendo que todos los elementos son distintos, caracterizad las entradas que producen los casos mejor y peor de *MergeSort* y de `QuickSort_min`. Indicad si dichas entradas son únicas y explicad el criterio de coste utilizado.
5. ¿Cuál de los dos algoritmos resulta más eficiente empíricamente en vuestras pruebas? Comparadlo con la predicción teórica. ¿Cuál requiere más memoria auxiliar y cómo depende esta del tamaño de entrada y de la estrategia de implementación? Justificad la respuesta.

Material que se debe entregar

La entrega se realizará a través de Moodle en un único fichero ZIP o TGZ que contenga:

1. La memoria; encontraréis una plantilla en Moodle.
2. Los ficheros fuente, las cabeceras y los demás archivos necesarios para compilar y reproducir los resultados.